

THE FIELD GUIDE · A TWO-HOUR LECTURE IN TEN PARTS

The PostGIS Almanac

**Spatial Databases & AI-Assisted GIS
for Public Health Research**



Press  /  to advance

2026 edition · PostGIS 3.6 · PostgreSQL 18 · GeoParquet · DuckDB · generative AI

What we'll cover

I. Overview, goals & how this runs

II. Why spatial databases?

III. Setup — your lab environment

HANDS-ON

IV. PostGIS fundamentals, in depth

V. Connecting & querying (SQL / Python / R)

VI. The modern geospatial stack (2026)

VII. Case study: clinic accessibility

HANDS-ON

VIII. Learning & doing GIS with generative AI

HANDS-ON

IX. Pop quiz

X. Summary & resources

Overall goal: Build real, hands-on fluency with PostGIS spatial databases for public health — set up a working lab, carry one analysis end-to-end, and learn to do GIS with generative AI as a copilot, responsibly.

PART I

Overview

The "where" of public health, why it needs better tools, and how today runs

Welcome — and how to follow along

This is a **hands-on** lecture. It's designed for researchers who may have limited prior experience with databases or programming, but who are familiar with public-health data.

We'll alternate **concept** → **demo** → **you try it**. Watch for these markers:

LAB Hands-on exercise

When you see this box, pause and run it yourself. Solutions follow.

Code is labelled by track:

SQL

PYTHON

R

DUCKDB

DOCKER

17

2026 edition: Updated for today's ecosystem — PostGIS 3.6, PostgreSQL 18, GeoParquet, DuckDB, `psycopy` v3, GeoPandas 1.x — plus a hands-on guide to learning and doing GIS with generative AI. New material is marked **NEW**.

Two ways to practice: a zero-setup **DuckDB** track that runs on any laptop, and a full **Docker PostGIS** track for the real multi-user server experience. We'll set up both in Part III.

Learning objectives

By the end of today, you should be able to:

- **Explain** why spatial databases beat CSVs and shapefiles for research
- **Stand up** a working PostGIS (Docker) and DuckDB spatial environment
- **Describe** geometry types, SRIDs/CRS, geometry vs. geography
- **Create** and use a spatial index — and read `EXPLAIN ANALYZE`
- **Write** spatial SQL: `ST_DWithin`, `ST_Intersects`, spatial joins, buffers
- **Connect** from Python (`geopandas`) and R (`sf`)
- **Carry** one public-health analysis end-to-end (clinic accessibility)
- **Use** generative AI to draft, translate, and debug GIS work — and verify it

The throughline: the *spatial SQL* you learn today transfers across PostGIS, DuckDB, and AI-assisted workflows alike. Learn it once; use it everywhere.

Today's agenda & timing

I Overview & goals ~5 min

II Why spatial databases ~10 min

III Setup — your lab ~15 min **HANDS-ON**

IV PostGIS fundamentals (deep) ~25 min

V Connecting & querying ~15 min

VI Modern stack ~10 min · then a 5-min break

VII Case study end-to-end ~25 min **HANDS-ON**

VIII GIS with generative AI ~20 min **HANDS-ON**

IX Pop quiz ~10 min

X Summary & resources ~5 min

Before we go further: if you can, start the Docker download now (Part III, slide with the `docker run` command) — it pulls in the background while we cover concepts.

PART II

Why Spatial Databases?

Spatial context, and the limits of flat files

Spatial data in public health

Spatial data links information to a specific **location on Earth**. In public health, understanding *where* health events occur, vulnerable populations live, and services sit is fundamental.

Examples

- Mapping disease outbreaks (influenza spread, cancer mortality by state)
- Hospital / clinic locations — geographic accessibility
- Patient addresses (anonymized / aggregated)
- Environmental exposure sources near communities
- Demographic data by census tract or ZIP code

Visualizing such data on maps reveals patterns and trends not apparent from tables alone — enabling **targeted, geographically-focused interventions**.

Key takeaway: Spatial context is crucial to understanding and addressing many public health issues.

The limits of traditional storage

CSVs and shapefiles are common — but they have real limits for large, complex, or collaborative work.



Challenges with CSVs

- **No spatial understanding** — coordinates are just numbers
- **Type ambiguity** — precision loss (ZIPs lose leading zeros)
- **Inconsistent formatting** — delimiters, special characters
- **Limited querying** — no spatial queries built in
- **Scalability** — poor with very large datasets
- **Integrity / security** — no validation or access control



Challenges with Shapefiles

- **File proliferation** — `.shp` `.shx` `.dbf` `.prj` together
- **Duplication / inconsistency** — many versions appear
- **Attribute limits** — 10-char fields, no true nulls, 2 GB cap
- **Limited analysis** — complex queries need external GIS
- **No concurrent access** — not built for multi-user editing

These undermine research quality, reproducibility, and collaboration. Relational databases (PostgreSQL) and spatial extensions (PostGIS) address them.

Traditional files vs. a database approach

Issue	With CSVs	With Shapefiles	Impact on research
Data integrity	Lat/lon as text, precision loss	Field names truncated; nulls → 0	Inaccurate analysis; can't tell missing from true zero
Scalability	Slow on 1 M address records	2 GB cap hit on national data	Can't analyze large / high-res data efficiently
Spatial querying	Can't query "clinics within 1 km"	Needs external GIS, complex steps	Slow, error-prone spatial questions
Collaboration	Overwrite risk on shared file	Hard to merge team edits	Hinders teamwork; data loss; reproducibility issues
Coordinate system	No embedded CRS info	<code>.prj</code> can be missing / wrong	Risk of misaligned spatial analysis

Key takeaway: Spatial databases offer a more robust, efficient, and powerful way to manage and analyze geographic health data — overcoming critical limits of file-based approaches.

The question we'll answer all day

To make this concrete, we'll carry **one realistic analysis** through the whole lecture.

RUNNING CASE STUDY

Clinic accessibility in Riverbend County

"How many people live within **5 km** of a primary-care clinic, and which **districts** are underserved relative to their population?"

Our (synthetic) data — three tables we'll build in Part III:

Table	Geometry	Key columns
clinics	Point (4326)	clinic_id, name
districts	MultiPolygon (4326)	district_id, name, population
blocks	Point (4326)	block_id, households, pop

We use **synthetic** data on purpose — never real patient locations in a teaching or AI context. More on that in Part VIII.

PART III

HANDS-ON

Setup — Your Lab Environment

Two tracks: DuckDB for everyone, Docker PostGIS for the full experience

Two tracks, one set of skills

You'll get the most out of today if you can run queries yourself. Pick a track — or do both.

DUCKDB Zero-setup, local

- One binary or `pip install` — **no server**
- Runs spatial SQL inside Python/R/CLI
- Reads GeoParquet & files directly
- Perfect for labs on any laptop

Best if you want to start immediately.

DOCKER Real PostGIS server

- A genuine multi-user PostgreSQL + PostGIS
- Concurrency, roles, indexes, `EXPLAIN`
- Mirrors a production research database
- Needs Docker Desktop installed

Best if you want the authentic workflow.

The SQL is nearly identical. DuckDB's `spatial` extension implements the same `ST_*` functions. Learn the query once; run it in either engine.

Track A — DuckDB in 60 seconds DUCKDB

The fastest path to running spatial SQL. No server, no accounts.

Option 1 — Python (what we'll use in labs)

```
pip install "duckdb>=1.1" "geopandas>=1.0"
```

```
import duckdb
con = duckdb.connect("riverbend.duckdb")
con.sql("INSTALL spatial; LOAD spatial;")
con.sql("SELECT ST_Point(0,0) AS p;").show()
```

Option 2 — CLI (great for quick checks)

```
# macOS / Linux
brew install duckdb # or download the binary
duckdb riverbend.duckdb
```

```
INSTALL spatial; LOAD spatial;
SELECT ST_AsText(ST_Point(-117.4, 33.9));
-- POINT(-117.4 33.9)
```

Lab 0a Verify DuckDB

Run either snippet. If you see a POINT printed, your spatial engine works. 

Track B — Docker PostGIS in one command

DOCKER

The official image bundles PostgreSQL 18 + PostGIS 3.6. One command, a real server.

```
# Pull & run a PostGIS server (downloads ~600 MB the first time)
docker run -d --name almanac-pg \
  -e POSTGRES_PASSWORD=almanac \
  -e POSTGRES_DB=riverbend \
  -p 5432:5432 \
  postgis/postgis:18-3.6
```

Connect with `psql` (inside the container):

```
docker exec -it almanac-pg \
  psql -U postgres -d riverbend
```

```
-- Turn on PostGIS in this database (once)
CREATE EXTENSION IF NOT EXISTS postgis;
SELECT postgis_full_version();
```

Tip: `-d` runs it in the background. Stop/restart with `docker stop almanac-pg` / `docker start almanac-pg`. Your data persists in the container until you `docker rm` it.

Teaching only: a hard-coded password and open port are fine on your laptop — never on a shared or internet-facing host.

Lab 0 — confirm your environment SQL

LAB 0B Prove PostGIS is alive

Run this in `psql` (Docker) or the DuckDB shell. Both should return a version string and a valid point.

DOCKER / POSTGIS

```
SELECT postgis_version();  
-- 3.6 USE_GEOS=1 USE_PROJ=1 ...  
  
SELECT ST_AsText(  
  ST_SetSRID(ST_MakePoint(-117.4, 33.9), 4326)  
);  
-- POINT(-117.4 33.9)
```

DUCKDB

```
LOAD spatial;  
SELECT version() -- v1.1+  
  
SELECT ST_AsText(  
  ST_Point(-117.4, 33.9)  
);  
-- POINT (-117.4 33.9)
```

Checkpoint: if both halves return a point, you're ready for everything that follows. Stuck? Flag it now — the rest of the lecture builds on this.

Loading the sample data SQL

We'll generate the synthetic Riverbend tables directly in SQL, so everyone has identical data.

Clinics — a handful of points

```
CREATE TABLE clinics (  
  clinic_id int PRIMARY KEY,  
  name text,  
  geom geometry(Point, 4326)  
);  
  
INSERT INTO clinics VALUES  
(1, 'Riverbend Family Health',  
  ST_SetSRID(ST_MakePoint(-117.40, 33.95), 4326)),  
(2, 'Eastside Community Clinic',  
  ST_SetSRID(ST_MakePoint(-117.31, 33.92), 4326)),  
(3, 'North Valley Health Center',  
  ST_SetSRID(ST_MakePoint(-117.36, 34.02), 4326));
```

Blocks — population points (generate a grid)

```
CREATE TABLE blocks AS  
SELECT  
  row_number() OVER () AS block_id,  
  (50 + (random()*450)):int AS pop,  
  ST_SetSRID(ST_MakePoint(  
    -117.45 + gx*0.01,  
    33.88 + gy*0.01), 4326) AS geom  
FROM generate_series(0, 18) gx,  
     generate_series(0, 16) gy;
```

~300 population points across the county. (DuckDB: use `range()` instead of `generate_series` in a cross join — the AI can translate it; see Part VIII.)

Lab oc Create the tables

Run both blocks. Then: `SELECT count(*) FROM blocks;` — expect ~323 rows.

Districts, and a reusable data pack

```
-- Districts: simple square cells acting as administrative areas
CREATE TABLE districts AS
SELECT
  (AS district_id,
   'District ' || AS name,
   8000 + random()*40000 :: int AS population,
   ST_SetSRID(
     ST_MakeEnvelope(
       -117.45 + (d%3)*0.10, 33.88 + (d/3)*0.08,
       -117.35 + (d%3)*0.10, 33.96 + (d/3)*0.08,
       4326) AS geom
  FROM generate_series(0,5) d;
```

You now have three layers: `clinics` (points), `blocks` (population points), and `districts` (polygons) — all in **SRID 4326**. This is the data for our case study in Part VII.

Prefer real files? The same tables ship as GeoParquet in the course pack. DuckDB can read them straight from a URL or disk — `SELECT * FROM 'clinics.parquet';` — no import step needed.

PART IV

PostGIS Fundamentals, In Depth

Geometry, coordinate systems, geography vs. geometry, indexing, and relationships

What are PostgreSQL and PostGIS?

PostgreSQL

A powerful, open-source **object-relational database**. Known for reliability, data integrity, scalability, and extensibility. Free and community-supported.

PostGIS

An **extension** that teaches PostgreSQL to store, manage, and analyze geographic objects. It adds:

- **Spatial data types** — `geometry`, `geography`
- **Spatial indexes** — R-Tree via GiST
- **Spatial functions** — hundreds, for distance, intersection, buffers...

Analogy: PostgreSQL is the filing cabinet; PostGIS adds special folders and tools for maps inside it.

Current (mid-2026): PostgreSQL **18** and PostGIS **3.6.x**. Recent versions expand raster, vector-tile (`ST_AsMVT`), and 3D support; modern PostgreSQL brings faster parallel queries and better JSON. You don't need the newest version to learn — but knowing what's current helps when reading docs and asking AI for help.

Key takeaway: PostGIS turns a very powerful database into a spatial powerhouse for geographic information.

Core concept 1 — Geometry types

PostGIS stores geographic features as specific geometry types.



Point

A single location (X, Y).

PH examples: patient residence, clinic location, disease case site, water sample point.



LineString

Connected sequence of points.

PH examples: roads, rivers, transit routes, disease-vector paths.



Polygon

A closed area.

PH examples: county / state boundaries, service areas, exposure zones, park boundaries.

PostGIS also supports **Multi-geometries** (MultiPoint, MultiLineString, MultiPolygon) — collections of the same type — and **GeometryCollections**.

How geometries are written — WKT & WKB

Every geometry has a human-readable and a binary form. You'll see both constantly.

WKT — *Well-Known Text* (readable)

```
POINT(-117.4 33.9)
LINESTRING(0 0, 1 1, 2 1)
POLYGON((0 0, 4 0, 4 4, 0 4, 0 0))
```

Constructing & inspecting

```
SELECT ST_AsText(          -- geometry → WKT
  ST_GeomFromText('POINT(-117.4 33.9)', 4326)
);
SELECT ST_AsGeoJSON(geom)  -- → GeoJSON for web maps
FROM clinics LIMIT 1;
```

WKB (*Well-Known Binary*) is the compact form actually stored on disk. You rarely write it by hand — functions convert for you.

Constructor cheat-sheet:

`ST_MakePoint(x, y)` — fast point

`ST_SetSRID(geom, 4326)` — stamp a CRS

`ST_GeomFromText(wkt, srid)` — from WKT

`ST_AsText` / `ST_AsGeoJSON` — read it back

Order matters: WKT and `ST_MakePoint` are **(longitude, latitude)** — X then Y. Reversing them silently puts your data in the wrong hemisphere.

Core concept 2 — Spatial reference systems (SRID)

An **SRS** / **CRS** defines how coordinate values map to real locations — the "language" or projection your data speaks.

- **Crucial for** aligning layers and meaningful measurements (distance, area)
- **SRID** — a unique integer ID per SRS (stored in `spatial_ref_sys`)
- **WGS84 (EPSG:4326)** — geographic lat/lon in degrees; used by GPS
- **Projected (UTM, State Plane)** — flatten Earth to a plane (meters/feet); better for local distance & area
- **Rule:** every geometry must have a defined SRID; layers must share one (or be transformed with `ST_Transform()`)

SRID alignment

Layer A · SRID X

Layer B · SRID Y

Misaligned ❌



Both → SRID Z

Aligned ✅

Layers with different SRIDs won't overlay unless transformed to a common system.

The classic trap — degrees are not meters

Distance on lat/lon (SRID 4326) is measured in **degrees**, which is almost never what you want.

```
-- WRONG: "0.045" — that's degrees, meaningless
SELECT ST_Distance(
  ST_SetSRID(ST_MakePoint(-117.40 33.95) 4326),
  ST_SetSRID(ST_MakePoint(-117.36 34.02) 4326)
);

-- RIGHT (option 1): transform to a metric CRS
SELECT ST_Distance(
  ST_Transform(a geom, 32611), -- UTM 11N, meters
  ST_Transform(b geom, 32611)
) FROM clinics a, clinics b
WHERE a clinic_id=1 AND b clinic_id=3;
-- ≈ 8100 (meters)
```

```
-- RIGHT (option 2): cast to geography (meters on a sphere)
SELECT ST_Distance(
  a geom::geography,
  b geom::geography
) FROM clinics a, clinics b
WHERE a clinic_id=1 AND b clinic_id=3;
-- ≈ 8120 (meters)
```

If a distance looks like **0.0x**, you're in degrees. Transform to a projected CRS (e.g. the right UTM zone) or cast to `geography`.

geometry vs. geography — which to use

PostGIS offers two spatial types. Choosing well saves you from the degrees-vs-meters trap.

`geometry` — Cartesian (flat)

- Math on a **flat plane** — fast
- Units are the CRS's units (degrees for 4326, meters for UTM)
- Hundreds of functions; the default for analysis
- **Best when** your data is in a projected CRS, or covers a small area

`geography` — geodetic (round Earth)

- Math on the **spheroid** — distances/areas in **meters**, always
- Correct over long distances & across UTM zones
- Fewer functions, somewhat slower
- **Best when** data spans large areas in lat/lon and you want true meters

Rule of thumb: national or continental lat/lon data and you want meters → `geography`. Local analysis or you've projected to UTM/State Plane → `geometry`. You can cast between them: `geom::geography`.

Core concept 3 — Spatial indexes

Like a book's index — but for **locations**. They dramatically speed up spatial queries ("what's near X?", "what's inside Y?").

- **Analogy:** a GPS that lets the database rapidly find spatial data
- **How (simplified):** PostGIS builds a **GiST** index over each geometry's **bounding box** (an R-Tree), so the planner skips most rows
- **Two-phase search:** a fast *index filter* on bounding boxes, then an *exact* check on the survivors

```
-- Build a spatial index (do this on every geometry column you query)
CREATE INDEX blocks_geom_idx ON blocks USING GIST (geom);
ANALYZE blocks;           -- refresh planner statistics
```

Functions that use the index automatically: `ST_DWithin`, `ST_Intersects`, `ST_Contains`, `&&` — because they begin with a bounding-box test. `ST_Distance` alone does *not*; prefer `ST_DWithin` for proximity.

Proving the index works — EXPLAIN ANALYZE

The planner tells you whether it used your index. Reading this is a core skill.

```
EXPLAIN ANALYZE
SELECT b.block_id
FROM blocks b, clinics c
WHERE c.clinic_id = 1
      AND ST_DWithin(
        b.geom, geography,
        c.geom, geography, 5000);
```

What to look for:

-  `Index Scan using blocks_geom_idx` — good, the index is used
-  `Seq Scan` on a big table — the index was skipped (missing index, wrong type, or a function wrapping the column)
- `actual time=` — real milliseconds; compare before/after adding the index
- `rows=` — estimate vs. actual; large gaps mean stale stats → run `ANALYZE`

Lab 1 Index & explain

Create `blocks_geom_idx`, run the `EXPLAIN ANALYZE` above, and note the time. Drop the index, re-run, and compare. (DuckDB users: skip — it auto-indexes; focus on the query.)

Spatial relationships — the vocabulary

Most spatial questions reduce to a handful of relationship predicates. They return `true` / `false`.

Function	Asks...
<code>ST_Intersects(a,b)</code>	do they touch/overlap at all?
<code>ST_Contains(a,b)</code>	is b fully inside a ?
<code>ST_Within(a,b)</code>	is a fully inside b ?
<code>ST_DWithin(a,b,d)</code>	are they within distance d ?
<code>ST_Touches(a,b)</code>	do they share only a boundary?
<code>ST_Crosses(a,b)</code>	does a line cross a polygon?

Under the hood these implement the **DE-9IM** model — a 3×3 matrix describing how two geometries' interiors, boundaries, and exteriors relate. You rarely write DE-9IM directly, but it's why the named predicates are precise and composable.

Measurement companions: `ST_Distance`, `ST_Length`, `ST_Area`, `ST_Buffer`, `ST_Intersection`, `ST_Union`, `ST_Centroid`.

The workhorse — spatial joins

A **spatial join** connects two tables by a spatial relationship instead of a shared key. This is where PostGIS earns its keep.

```
-- Count population points inside each district
SELECT d.name,
       count(b.*) AS n_blocks,
       sum(b.pop) AS people
FROM districts d
JOIN blocks b
  ON ST_Contains(d.geom, b.geom) -- the join!
GROUP BY d.name
ORDER BY people DESC;
```

How it works:

- The `ON` clause uses a **spatial predicate**, not `a.id = b.id`
- The planner uses the GiST index to avoid comparing every pair
- `JOIN` keeps matches; `LEFT JOIN` keeps districts with **zero** points too (watch for those!)

Pattern to memorize: *points-in-polygons* via `JOIN ... ON ST_Contains(poly, pt) + GROUP BY`. It answers a huge share of public-health questions.

Part IV — key takeaway

PostGIS stores shapes (geometries) with a real-world coordinate system (SRID), lets you choose `geometry` (flat, fast) or `geography` (true meters), uses **GiST spatial indexes** to stay fast, and answers questions through **relationship predicates and spatial joins**.

Watch the two classic traps: **degrees vs. meters**, and **queries that skip the index** (`EXPLAIN ANALYZE` is your friend).

PART V

Connecting & Querying

Reaching PostGIS from SQL, Python, and R

General connection principles

To reach any database, you need five standard parameters:

Parameter	Meaning
Host	Server address (<code>localhost</code> , IP, domain)
Port	Endpoint — PostgreSQL default <code>5432</code>
Database	The specific db to connect to
Username	Your database user
Password	Password for that user

For demos and AI work, use a **read-only account** to prevent accidental modification of data.

Key takeaway: These five pieces of information are the standard keys to unlock almost any database. For our Docker server: host `localhost` , port `5432` , db `riverbend` , user `postgres` , password `almanac` .

A deeper note on SQL SQL

PostGIS is SQL — the same `SELECT ... FROM ... WHERE ... GROUP BY` you may know, plus spatial functions in the mix.

```
SELECT           -- columns / computed values
  d.name,
  count(b.*)      AS n,
  sum(b.pop)      AS people
FROM districts d  -- a table
JOIN blocks b    -- joined to another...
  ON ST_Contains(d.geom, b.geom) -- ...spatially
WHERE d.population > 10000 -- filter rows
GROUP BY d.name          -- aggregate
HAVING sum(b.pop) > 0     -- filter groups
ORDER BY people DESC     -- sort
LIMIT 10;               -- cap output
```

Spatial functions slot in **anywhere an expression is**

allowed — in `SELECT`, `WHERE`, `JOIN ... ON`, even `ORDER BY`.

```
ST_Distance(a, b) -- distance
ST_DWithin(a, b, d) -- within d?
ST_Intersects(a, b) -- overlap?
ST_Contains(a, b) -- b inside a?
ST_Buffer(g, d) -- grow a zone
```

Doing spatial work **in the database** means the server filters before sending data to R/Python — often far faster.

Connecting from Python —

geopandas + **psycopg v3** PYTHON

```
# pip install "geopandas>=1.0" "psycopg[binary]" "sqlalchemy>=2" geoalchemy2
# Note (2026): psycopg v3 is current; psycopg2 is legacy / maintenance-only.
import geopandas as gpd
from sqlalchemy import create_engine

# Build a SQLAlchemy engine — "+psycopg" selects the v3 driver
user, pw = "postgres", "almanac"
host, port, db = "localhost", 5432, "riverbend"
engine = create_engine(f"postgresql+psycopg://{user}:{pw}@{host}:{port}/{db}")

# Read a spatial query straight into a GeoDataFrame
sql = """SELECT clinic_id, name, geom
        FROM clinics;"""
gdf = gpd.read_postgis(sql, engine, geom_col='geom')

print(gdf.head())      # gdf.explore() -> interactive Leaflet map

engine.dispose()       # good practice in longer scripts
```

Key takeaway: with **geopandas** + SQLAlchemy, Python works with PostGIS intuitively — especially if you know **pandas**. The

Same skills, no server — DuckDB from Python

DUCKDB

PYTHON

The identical analysis, running in-process over your local tables or files.

```
import duckdb, os.path as op

con = duckdb.connect("riverbend.duckdb")
con.sql("LOAD spatial;")

# The SAME spatial SQL you'd run in PostGIS
r = con.sql("""
SELECT d.name, sum(b.pop) AS people
FROM districts d
JOIN blocks b ON ST_Contains(d.geom, b.geom)
GROUP BY d.name ORDER BY people DESC
""").ff()

# Read big open data with zero import:
con.sql("SELECT count(*) FROM 'overture_places.parquet'")
```

Notice: the `ST_Contains` spatial join is byte-for-byte the same as the PostGIS version. *That's the payoff of learning spatial SQL once.*

Supplement — connecting from R

For R users, `sf` + `DBI` + `RPostgres` mirror the Python workflow exactly.

```
# install.packages(c("sf","DBI","RPostgres")) # once
library(sf) library(DBI) library(RPostgres)

con <- dbConnect(RPostgres::Postgres(),
  dbname = "riverbend",
  host   = "localhost",
  port   = 5432,
  user    = "postgres",
  password = "almanac"

clinics <- st_read(con, query = "SELECT clinic_id, name, geom FROM clinics;")

print(clinics) # plot(st_geometry(clinics))
dbDisconnect(con) # always close
```

One mental model, three dialects: SQL is the engine; `geopandas` and `sf` are thin, friendly wrappers. Pick the language you know — the spatial SQL is shared.

Lab 2 — connect & run a spatial join

PYTHON

SQL

LAB 2 Your first end-to-end query

1. Connect from Python (PostGIS engine *or* DuckDB connection).
2. Run the population-per-district spatial join.
3. Print the busiest district. Bonus: `gdf.explore()` to see it on a map.

```
# Works against either engine — just change how you read:
sql = """SELECT d.name, sum(b.pop) AS people
        FROM districts d
        JOIN blocks b ON ST_Contains(d.geom, b.geom)
        GROUP BY d.name ORDER BY people DESC"""

# PostGIS: df = gpd.read_postgis(sql.replace(' geom',''), engine) # no geom needed here
# DuckDB : df = con.sql(sql).df()
print(df.head(1))    # -> the most-populated district
```

Checkpoint: you've now connected, run a spatial join, and pulled results into a dataframe — the core loop of every analysis to come.

PART VI

NEW

The Modern Geospatial Stack

2026: think in terms of a stack, not a single tool

PostGIS is no longer the only place for spatial SQL

A major shift: you can now run powerful spatial SQL **without a database server at all**.

The biggest example is **DuckDB** with its `spatial` extension — an in-process analytical engine ("SQLite for analytics") that speaks much of the same `ST_*` SQL you learn for PostGIS.

- **No server** to install or manage — runs inside R, Python, or the CLI
- Reads / writes **GeoParquet**, GeoJSON, shapefiles; queries files on disk or in cloud storage (S3/GCS/Azure) directly
- Extremely fast over tens-to-hundreds of millions of features on a laptop

Reach for DuckDB: ad-hoc analysis, one-off transforms, reading big open-data files, reproducible notebook pipelines.

PostGIS is still better for: a shared, central, multi-user database; enforced integrity; transactional editing; a long-lived authoritative dataset.

You can use them together — `pg_duckdb` and DuckDB's `postgres` extension let DuckDB query a live PostGIS database.

Cloud-native geospatial formats

The "shapefile era" is fading. Modern formats are built for the cloud, for big data, and for streaming only the bytes you need.

GeoParquet

Columnar, compressed vector format — fast, tiny, the emerging standard for sharing large vector datasets. Read by GeoPandas, DuckDB, QGIS. A modern replacement for shapefiles / CSV exports.

PMTiles & vector tiles

A single-file format for serving interactive maps cheaply (even from static hosting). PostGIS generates vector tiles directly with `ST_ASMVT`.

COG & STAC (raster)

Cloud-Optimized GeoTIFFs stream just the pixels you need from imagery; STAC is the catalog standard for finding it — relevant for environmental-exposure work.

Overture Maps

A large, openly-licensed dataset (places, buildings, transport, admin areas) distributed as GeoParquet — free context for health analyses.

Indexing beyond R-Trees, and choosing a tool

H3 — discrete global grids

Hexagonal-cell grid systems like **H3** are popular for aggregating and joining point data at scale. The `h3-pg` extension brings H3 into PostGIS — handy for **privacy-preserving aggregation** of patient locations into uniform cells.

Your situation	Default tool
Shared, authoritative, many analysts	PostGIS
Ad-hoc analysis of big files on a laptop	DuckDB + spatial
Interactive dataframe workflow	GeoPandas / sf
Sharing / archiving a dataset	GeoParquet
Publishing an interactive web map	PMTiles / ST_AsMVT

Key takeaway: Think in terms of a *stack*. PostGIS is your durable, shared database; DuckDB + GeoParquet make ad-hoc and cloud analysis fast and cheap; GeoPandas / sf remain your interactive home. The spatial SQL you learn transfers across all of it.



5-minute break. When we return: we build the full clinic-accessibility analysis end-to-end on your data.

PART VII

HANDS-ON

Case Study — Clinic Accessibility

One question, carried end-to-end on your own database

The analysis, start to finish

We'll answer our running question in **five steps**, each a short lab. By the end you'll have a reusable accessibility pipeline.

1 Inventory & sanity-check the data — counts, SRIDs, validity

2 Proximity — population within 5 km of any clinic

3 Service areas — buffers and what they cover

4 Aggregate by district — access rate per population

5 Validate — sanity checks before we trust it

Question: "How many people live within 5 km of a clinic, and which districts are underserved relative to population?" Every step is runnable in PostGIS *or* DuckDB.

Step 1 — inventory & sanity-check SQL

Before any analysis: confirm what you have. Counts, coordinate systems, and validity.

```
-- How much data, and what SRID?
SELECT 'clinics', count(*), ST_SRID(geom) FROM clinics GROUP BY 3
UNION ALL
SELECT 'blocks', count(*), ST_SRID(geom) FROM blocks GROUP BY 3
UNION ALL
SELECT 'districts', count(*), ST_SRID(geom) FROM districts GROUP BY 3

-- Any invalid polygons? (real data often has them)
SELECT district_id
FROM districts
WHERE NOT ST_IsValid(geom);
```

What we're checking:

- **All three layers share SRID 4326** → they'll align without transforming
- **Row counts** match what we loaded (~3 / ~323 / 6)
- **No invalid geometries** — if any, fix with `ST_MakeValid(geom)`

Lab 3.1

Run both queries. Confirm matching SRIDs and zero invalid rows before continuing.

Step 2 — population within 5 km of a clinic SQL

The core accessibility measure: which population points are "served"?

```
-- Mark each block as served / unserved, using true meters via geography
SELECT
  sum(pop) AS total_pop,
  sum(pop) FILTER (WHERE served) AS served_pop,
  round(100.0 * sum(pop) FILTER (WHERE served) / sum(pop), 1) AS pct_served
FROM (
  SELECT b.pop,
         EXISTS (
           SELECT 1 FROM clinics c
           WHERE ST_DWithin(b.geom, geography(c.geom), 5000)
         ) AS served
  FROM blocks b
)
```

Why `ST_DWithin` + `::geography`: it uses the spatial index *and* measures real meters — both traps from Part IV, avoided in one line.

Lab 3.2

Run it. What % of the county is within 5 km of a clinic? Try 3 km and 10 km — how sensitive is access to the threshold?

Step 3 — service areas as buffers SQL

Sometimes you want the **zone** itself — to map it, or intersect it with other layers.

```
-- A 5 km service area around each clinic
SELECT clinic_id,
       ST_Buffer(geom::geography, 5000)::geometry AS svc
FROM clinics;

-- Union them into one combined coverage polygon
SELECT ST_Union(
       ST_Buffer(geom::geography, 5000)::geometry
) AS coverage
FROM clinics;
```

- `ST_Buffer` on `geography` grows a true-meter zone
- `ST_Union` dissolves overlapping buffers into one coverage shape
- Export `coverage` as GeoJSON to drop onto a web map, or intersect it with districts (next step)

Buffers are approximations of circles (segmented polygons) and ignore roads/terrain. For travel-time access you'd use a routing service — note the assumption.

Step 4 — access rate by district SQL

Now the decision-relevant output: a **rate per district**, so we compare fairly across populations.

```
SELECT
  d.name
  ,d.population
  ,sum(b.pop) AS pop_in_blocks
  ,sum(b.pop) FILTER (WHERE served) AS served_pop
  ,round(100.0 * sum(b.pop) FILTER (WHERE served)
        / nullif sum(b.pop) 0,1) AS pct_served
FROM districts d
JOIN blocks b ON ST_Contains(d.geom, b.geom) -- points-in-polygons
CROSS JOIN LATERAL (
  SELECT EXISTS (SELECT 1 FROM clinics c
    WHERE ST_DWithin(b.geom::geography, c.geom::geography, 5000)) AS served
) x
GROUP BY d.name, d.population
ORDER BY pct_served ASC -- worst-served first
```

Lab 3.3

Run it. Which two districts are **least** served? Those are where you'd target a new clinic.

Step 5 — validate before you trust it SQL

A number is not an answer until you've checked it. Build the habit now.

Cross-checks:

- **Totals reconcile** — does `sum(served) ≤ total_pop`? Does district pop roughly match `population`?
- **Spot a known case** — pick one block next to a clinic; is it flagged served?
- **Boundary effects** — blocks near the county edge may have clinics *just outside* your data → undercount
- **Units** — distances in meters? `geography` confirms it
- **Sensitivity** — does the ranking hold at 3 km vs 5 km?

-- One block you can reason about by hand

```
SELECT b.block_id,  
       ST_Distance(b.geom::geography,  
                  c.geom::geography)::int AS m  
FROM blocks b  
JOIN clinics c ON c.clinic_id = 1  
ORDER BY m  
LIMIT 3;      -- nearest blocks to clinic 1
```

Validation is the analyst's job — not the database's, and certainly not the AI's.

Extending the pattern — two more public-health questions

The same building blocks answer many questions. Notice the recurring verbs: *contain*, *within-distance*, *buffer*, *intersect*, *aggregate*.



Disease surveillance & clusters

Cases (points) in districts (polygons), normalized by population.

```
SELECT d.name, count(c.*) AS cases
FROM districts d
JOIN cases c ON ST_Contains(d.geom, c.geom)
GROUP BY d.name -- ÷ population → rate
```



Environmental exposure

Who lives inside a pollutant buffer?

```
SELECT sum(b.pop) AS exposed
FROM blocks b, sources s
WHERE ST_DWithin(
  b.geom, geography(
    s.geom, geography(2000) -- 2 km
```

Key takeaway: PostGIS answers sophisticated spatial questions efficiently — and once you know the handful of core predicates, new questions are recombinations of the same moves.

PART VIII

HANDS-ON

Learning & Doing GIS with Generative AI

A tutor, translator, debugger, and copilot — used responsibly

The mindset, and four ways AI helps

Mindset: treat the AI as a tireless, fast, occasionally-overconfident teaching assistant and pair-programmer — *not* an oracle. It excels at drafting, explaining, and translating. *You* remain the analyst responsible for correctness, ethics, and interpretation.

1. Tutor & explainer 📖

Explains concepts at your level with public-health examples — SRIDs, geometry vs. geography, spatial joins, "why is my distance in degrees?".

2. Code generator & translator ↻

Turns plain English into spatial SQL, or translates the same analysis between SQL, R (`sf`), and Python (`geopandas`).

3. Debugger & reviewer 🐛

Paste an error and your query; ask what's wrong, how to fix it, and whether it's using the spatial index.

4. Analysis copilot 🚀

Plans a whole analysis — what data, which functions, how to validate. Coding agents can even run code and iterate with you.

How to write a good GIS prompt

The answer's quality depends heavily on the context you give. A reliable recipe:

- **Role & level** — "You are a PostGIS expert helping a public-health researcher new to SQL."
- **Environment** — tool & version ("PostGIS 3.6, PostgreSQL 18"); language (SQL / R `sf` / Python `geopandas`)
- **The data** — table names, key columns, geometry column & type, and the **SRID** of each layer
- **The goal** — the precise question and the **units** you want (meters? rate per 1,000?)
- **The ask** — "explain each step", "use a spatial index", "keep it to one query"

GENERATE SPATIAL SQL

You are a PostGIS expert (PostGIS 3.6 / PostgreSQL 18). I have: - "cases": id, diagnosis_date, geom (Point, SRID 4326) - "districts": district_id, name, population, geom (MultiPolygon, SRID 4326) Write one query returning each district's name, case count, and cases per 1,000 population. Make sure it can use a spatial index, and explain each line.

Lab 4 — drive the AI, then verify it

PYTHON

SQL

LAB 4 Prompt → run → verify

Using your own Riverbend schema, ask an AI assistant to write the **5 km access rate by district** query (our Step 4). Then **verify** it against the answer you already have.

YOUR PROMPT

You are a PostGIS 3.6 expert helping a public-health researcher. Tables (all SRID 4326): - clinics(clinic_id, name, geom Point) - blocks(block_id, pop, geom Point) - districts(district_id, name, population, geom Polygon) Write ONE query: per district, the % of block population within 5 km of any clinic. Use meters, use the spatial index, and explain each line.

Then check the AI's output against reality:

1. Does it use `::geography` or `ST_Transform` (meters, not degrees)?
2. Does it use `ST_DWithin` (index) — not bare `ST_Distance`?
3. Run it. Does the ranking **match your Step 4 result**?
4. Ask a follow-up: "add `EXPLAIN ANALYZE` and confirm the index is used."

If its numbers differ from yours, **you** find out why. That gap-hunting is the skill.

Copy-ready prompt templates

Fill in the `[brackets]` and paste into your AI assistant of choice.

EXPLAIN A CONCEPT

You are a patient GIS tutor. Explain `[CONCEPT]` to a public-health researcher who knows statistics but is new to spatial data. Use a concrete clinic/disease-mapping example, define jargon, and finish with one common mistake to avoid.

TRANSLATE BETWEEN LANGUAGES

Here is a PostGIS SQL query: `[PASTE]`. Show the equivalent (a) in R using `sf` and (b) in Python using `geopandas`, reading from the same database. Note any behavioural differences.

DEBUG AN ERROR / EMPTY RESULT

My PostGIS query returns `[ERROR / "0 rows but I expected matches"]`. Query: `[PASTE]`. SRIDs: `[LIST]`. Geometry columns: `[LIST]`. What are the most likely causes, ranked, and how do I check and fix each?

PLAN & SANITY-CHECK

I want to `[GOAL, e.g. "estimate people within 5 km of a clinic in County X"]`. I have `[DATA]`. Propose a step-by-step PostGIS workflow, flag CRS/units assumptions, and give three checks to confirm the results are sensible.

A safe AI-assisted workflow

Use AI as a **loop**, not a **one-shot**:

1 Frame the question yourself. Know what answer would make sense — rough magnitude, units, expected shape. Your guardrail against confident-but-wrong output.

2 Give rich context. Tool / version, table & column names, geometry types, SRIDs.

3 Ask for an explanation, not just code. "Explain each step" surfaces flawed assumptions.

4 Run on a small sample first. Use `LIMIT`, one district, or a known case you can predict.

5 Verify against reality. Check counts and a few records by hand; cross-check function names in the official docs.

6 Iterate. Feed back what you saw ("totals are 10× too high — a unit issue?") and refine.

Trust, but verify — common AI pitfalls in GIS

⚠️ **AI can be confidently wrong.** Watch for these GIS-specific failure modes:

- **Invented functions/args** — confirm names in the [PostGIS reference](#)
- **SRID / units mistakes** — distance on SRID 4326 is in *degrees*; remember `ST_Transform` or the `geography` type
- **Invalid geometries** — real data self-intersects; run `ST_IsValid` / `ST_MakeValid`
- **Performance blind spots** — a "correct" query can be unusably slow; ask for `EXPLAIN ANALYZE` and index use
- **Outdated patterns** — it may suggest `psycopg2` instead of `psycopg` v3; state your versions
- **Statistical subtlety** — MAUP, edge effects, ecological fallacy are easy to gloss over; apply your expertise

You met every one of these today: degrees vs. meters (Step 2), index use (Lab 1), boundary effects (Step 5). That's exactly the checklist to run on AI output.



Data privacy & ethics (read this twice)

Never paste protected health information (PHI/PII) or precise patient locations into a consumer AI chat.

Patient addresses and coordinates are among the most re-identifiable data that exist — pasting them into a public tool may violate HIPAA, GDPR, IRB protocols, and data-use agreements.

- **Share schema, not data** — give the AI table/column structure, types, and SRIDs, never real rows
- **Use synthetic examples** when you need sample values (exactly what we did with Riverbend today)
- **De-identify and aggregate** first — to census tracts or H3 cells; apply small-cell suppression
- **Prefer approved / enterprise AI** (zero-retention, no-training) or local models near sensitive data — follow institutional policy first
- **Keep a human in the loop** — AI informs analysis; it does not make public-health decisions

Good practice: "I have a points table of de-identified case locations (SRID 4326) and a polygons table of census tracts; help me aggregate cases per tract" — gives the AI everything it needs *without exposing a single real record*.

The AI tooling landscape, and emerging applications

NEW

Tooling landscape

-  **Chat assistants** — Claude, ChatGPT, Gemini: explain, draft, translate, debug
-  **In-editor** — Copilot, Cursor, notebook AI: autocomplete & refactor in place
-  **Coding agents** — Claude Code reads files, runs queries, iterates end-to-end
-  **MCP & text-to-SQL** — connect AI to a *read-only* PostGIS to query in natural language (scope permissions tightly!)

Emerging GenAI in the field

-  **Geospatial foundation models** — Prithvi, Clay: flood/burn mapping from imagery
-  **Promptable extraction** — SAM-for-geo ([samgeo](#)): buildings/water by click or prompt
-  **"Chat with your map"** — QGIS LLM plugins generate geoprocessing / SQL
-  **Synthetic data** — privacy-preserving populations for sharing & teaching
-  **Semantic + spatial search** — [pgvector](#) combines embeddings with spatial filters
-  **Agentic copilots &  AI cartography**

⚠ **Same rules apply:** foundation-model outputs need ground-truthing, generated code/SQL needs verification, and **none of this excuses putting real PHI into unapproved tools.** Treat AI output as a draft to validate, not a result to publish.

Part VIII — key takeaway

Generative AI dramatically lowers the barrier to learning and doing GIS — as a tutor, code translator, debugger, and analysis copilot.

Get the most from it by giving **rich context** (versions, schema, SRIDs), asking for **explanations**, and **verifying every result** — exactly the validation habits you practiced in the case study. And never trade your participants' privacy for convenience: **share structure, not sensitive data.**

PART IX

Pop Quiz

Click to reveal each answer — test your recall

Q1 • Q2

Q1. A key advantage of PostGIS over shapefiles for *collaborative* research?

A) Shapefiles support more advanced SQL B) **PostGIS allows robust concurrent access & integrity** C) Shapefiles handle large datasets better D) PostGIS needs no coordinate systems

Answer: B. A database is built for concurrent multi-user access and enforces integrity — critical for collaboration. Shapefiles are prone to versioning issues.

Q2. Best geometry type for administrative district boundaries (counties)?

A) Point B) LineString C) **Polygon** D) MultiPoint

Answer: C. Polygons represent areas with defined boundaries — the right choice for counties or states.

Q3 · Q4

Q3. The primary purpose of an SRID?

- A) Uniquely identify each record
- B) Define map-feature styling
- C) **Define the coordinate system & give real-world meaning**
- D) Speed up non-spatial attribute queries

Answer: C. The SRID defines the CRS (e.g. WGS84, UTM), essential for interpreting coordinates and aligning layers.

Q4. Your `ST_Distance` on SRID-4326 data returns `0.045`. What went wrong?

- A) The geometries are invalid
- B) **It's in degrees — transform to a metric CRS or cast to `geography`**
- C) The spatial index is missing
- D) Nothing; 0.045 km is correct

Answer: B. On 4326, distance is in degrees. Use `ST_Transform` to a projected CRS or cast to `geography` for true meters.

Q5 · Q6

Q5. To find clinics within 2 km of an address *efficiently*, the best function?

A) `ST_Area()` B) `ST_DWithin()` C) `ST_Union()` D)

`ST_Distance()` in a `WHERE`

Answer: B. `ST_DWithin(a, b, distance)` tests proximity *and* leverages the spatial index. A bare `ST_Distance` filter cannot use the index.

Q6. Fast, one-off analysis of a 50-M-row GeoParquet file on your laptop, no server?

A) A shapefile in a spreadsheet B) **DuckDB with the spatial extension** C) Emailing the file to a colleague D) Converting everything to CSV first

Answer: B. DuckDB + `spatial` needs no server, reads GeoParquet directly, and runs spatial SQL fast over tens of millions of rows.

Q7 • Q8

Q7. Using a generative AI assistant on patient case locations — most appropriate practice?

- A) Paste the full coordinate table for full context B) **Describe the schema (tables/columns, types, SRIDs) + synthetic examples — never real PHI** C) Trust the query without checking D) Skip stating your PostGIS version

Answer: B. Share *structure, not sensitive data*. Schema + SRIDs + synthetic examples give the AI what it needs without exposing re-identifiable data — and you still verify.

Q8. A spatial join counting points in polygons is typically written as...

- A) `JOIN ON a.id = b.id` B) `JOIN ON ST_Contains(poly.geom, pt.geom)` + `GROUP BY` C) Two separate queries merged in a spreadsheet D) `ORDER BY ST_Area(geom)`

Answer: B. Spatial joins relate tables by a spatial predicate, not a shared key — the points-in-polygons pattern you used in the case study.

PART X

Summary & Resources

Quick summary — what you can now do

- **Set up a lab** — Docker PostGIS and/or DuckDB spatial, from one command
- **Reason about fundamentals** — geometry types, SRID/CRS, geometry vs. geography, indexes
- **Avoid the classic traps** — degrees vs. meters; queries that skip the index (`EXPLAIN ANALYZE`)
- **Write spatial SQL** — predicates, buffers, and the points-in-polygons spatial join
- **Connect** from SQL, Python (`geopandas` / `psycopg` v3), and R (`sf`)
- **Carry an analysis end-to-end** — the clinic-accessibility pipeline, with validation
- **Use the modern stack** — GeoParquet, DuckDB, Overture; the same SQL transfers
- **Work with generative AI** — draft, translate, debug — with rich context, verification, and privacy

Bottom line: PostGIS is a durable, shared spatial foundation — and the spatial SQL you learned today carries across the whole modern, AI-assisted ecosystem.

Resources for further learning

Official

- PostGIS — postgis.net
- PostgreSQL — postgresql.org
- Spatial SQL reference — postgis.net/docs/reference.html
- PostGIS Docker image — hub.docker.com/r/postgis/postgis

Python & R

- `geopandas` — geopandas.org
- `psycopg` — psycopg.org/docs
- `sf` — r-spatial.github.io/sf

Modern stack

NEW

- DuckDB Spatial — [duckdb.org → spatial](https://duckdb.org/spatial)
- GeoParquet — geoparquet.org
- Overture Maps — overturemaps.org
- *Spatial SQL* (Forrest book) & PostGIS in Action

GenAI in geospatial

NEW

- Prithvi — ibm-nasa-geospatial
- Clay — madewithclay.org
- `segment-geospatial` — samgeo.github.org
- `pgvector` — github.com/pgvector/pgvector

THE POSTGIS ALMANAC

Thank you



Keep the momentum: re-run the clinic-accessibility case study on your own data, with an AI assistant as your tutor — and verify every result.

Always state your tool + version, share schema & SRIDs (not PHI), verify every result.

Built with [Slidev](#)